

Object-Oriented Programming in Java



OOP (Object-Oriented Programming) is a programming paradigm that focuses on **creating and modeling real-world entities as objects**.

Each object contains its **own properties and behaviours**.

In OOP, programs are designed **by combining objects** that interact with each other to **perform tasks**.

In OOP, the basic building blocks are:

- ↳ Classes
- ↳ Objects
- ↳ Constructors

The core principles of OOP are:

- ↳ Inheritance
- ↳ Polymorphism
- ↳ Encapsulation
- ↳ Abstraction

↳ **Classes:**



A class is a **blueprint, template, or user-defined type** that **combines properties and behaviours** into a **single unit**.

A **single unit** means a **class** groups **attributes and methods** together within the **same structure**.

Syntax:

```
Class ClassName {  
    //statements  
}
```

Note: In Java the **filename must** be the same as the **public class name** to ensure the compiler can correctly map the **java class to its source file**.

↳► Objects:



An object is an **instance of a class** that represents a **real-world entity**.

In Java, Every object has its own unique **properties and behaviours**.

In Java, we use the **new** keyword to create an object.

Syntax: - ClassName **referenceName** = **new** ConstructorCall();

A **constructor call** invokes the **constructor to initialize** the newly created **object**.

Example:



```
1 package com.channa.JavaPractice; //package
2 public class Demo {
3     Run | Debug
4     public static void main(String[] args) {
5         Demo myDemo = new Demo();
6         //className Reference = new ConstructorCall();
7     }
8 }
```

↳► Constructors:



A constructor is a **special member** of a class because it has **no return type**, **must have same name as the class**, it is **automatically called** when an **object is created**, constructor **cannot be static**, and is **used to initialize objects** and ensure they are **created in a valid state**.

Note: Constructors are treated as **special members** because **they don't follow the normal rules of methods**.

In Java, constructors are classified into three main types:

1. Parameterized constructor
2. No-argument constructor
3. Default constructor

Note: We will learn about them after learning methods and variables.

Methods in Java



A **method** is a **reusable block of code** used to **perform specific operations** based on its **implementation**.

In Java, **methods** are classified into **2 types**:

1. **Static methods**
2. **Instance methods**

➤ Static method:



A **static method** is a **method** defined inside a class using the **static** keyword.

A **static method** belongs to the **class**. It allows a **method** to be called **without creating an instance of the class** and is primarily used to **create utility or helper methods**.

Syntax: **static returnType** methodName(**parameters**) {
 // method body
 }

Example:

```
1 package com.channa.JavaPractice;//package
2 public class Demo {
3     Run | Debug
4     public static void main(String[] args) {
5         // main method itself is a static method.
6         // It allows the JVM to call it without creating an instance of the class.
7     }
8 }
```

↳ Instance method:

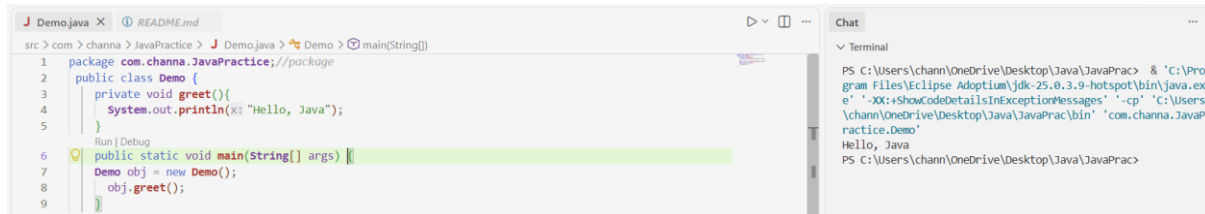


An **instance method** is a **method** defined inside a class **without the static keyword**.

An **instance method** belongs to an **instance of the class**, its purpose is to **perform operations on the state and behaviour of an instance (object)**.

Syntax: **returnType** **methodName(parameters)** {
 // method body
 }

Example:



```
1 package com.channa.JavaPractice; //package
2 public class Demo {
3     private void greet(){
4         System.out.println("Hello, Java");
5     }
6     public static void main(String[] args) {
7         Demo obj = new Demo();
8         obj.greet();
9     }
}
```

Terminal output: PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'

Variables in Java



A **variable** is a **named memory container** used to **store data** that can be **accessed and manipulated** during **program execution**.

In Java, **variables** are classified into **3 types**:

1. Local variable
2. Static variable
3. Instance variable

➤ **Local variable:**



Local variables are **declared and initialized** within a **method, constructor, or block**.

They are **accessible only within that method, constructor, or block** and do not have **default values**, so **initialization is mandatory before use**.

Example:



```
1 package com.channa.JavaPractice; //package
2 public class Demo {
3     public static void main(String[] args) {
4         int a;
5         System.out.println(a); //CTE
6         int b = 10;
7         System.out.println(b); //10
8     }
9 }
```

➤ Static variable:



Static variable is declared inside a class, outside any method, constructor, or block, using the **static** keyword.

It is also known as a **class-level variable** because it **belongs to the class itself**, not to any **specific object**.

Static variables can be accessed using the **class name**, followed by the **dot (.) operator** and the **variable name**.

Static variables are initialized and allocated memory only once when the class is loaded and exist until the class is unloaded.

Syntax: dataType variableName;

Example:

```
1 package com.channa.JavaPractice; //package
2 public class Demo {
3     static String university = "Garden City University"; // static variable
4     public static void main(String[] args) {
5         System.out.println(Demo.university); // classname is not mandatory but recommended
6     }
7 }
```

Terminal Output:

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
Garden City University
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

Note: We use **static variables** because they provide a **common point of access** to shared data across all **instances of a class**.

➤ Instance variable:



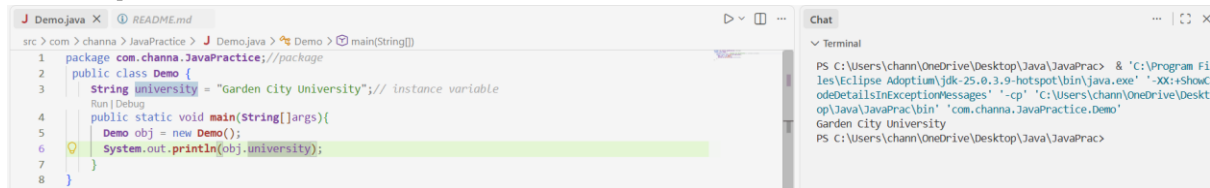
Instance variable is declared inside a class, outside any method, constructor, or block, without the **static** keyword.

To access an **instance variable**, **object creation** is mandatory.

Instance variables can be accessed using the **instance reference variable name**, followed by the **dot (.) operator** and the **variable name**.

Syntax: dataType variableName;

Example:



```
1 package com.channa.JavaPractice; //package
2 public class Demo {
3     String university = "Garden City University"; // instance variable
4     public static void main(String[] args) {
5         Demo obj = new Demo();
6         System.out.println(obj.university);
7     }
8 }
```

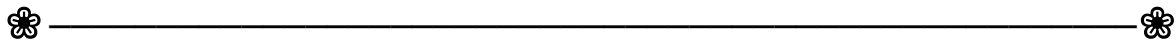
Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
Garden City University
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

We use a **non-static (instance) variable** when **each object needs its own copy of the data** and should be **independent of other objects**.

The objects are independent because changing the **attribute of one object** does not affect the **attribute of another object**.

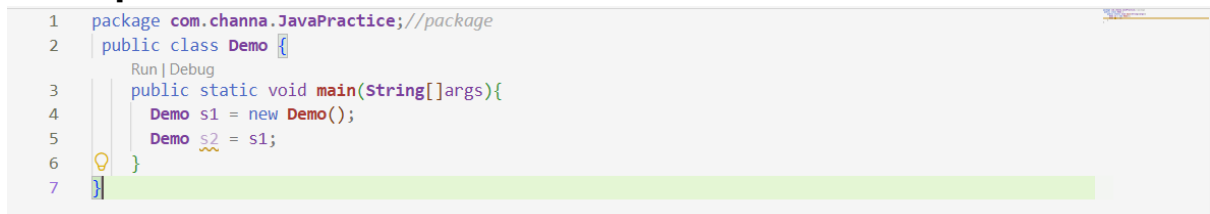
➤ Reference Copy:



Reference copy in Java is when you assign **one object reference to another** using the **simple assignment operator (=)**.

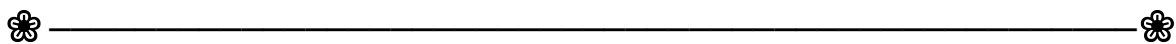
Both **references point to the same object in memory**.

Example:



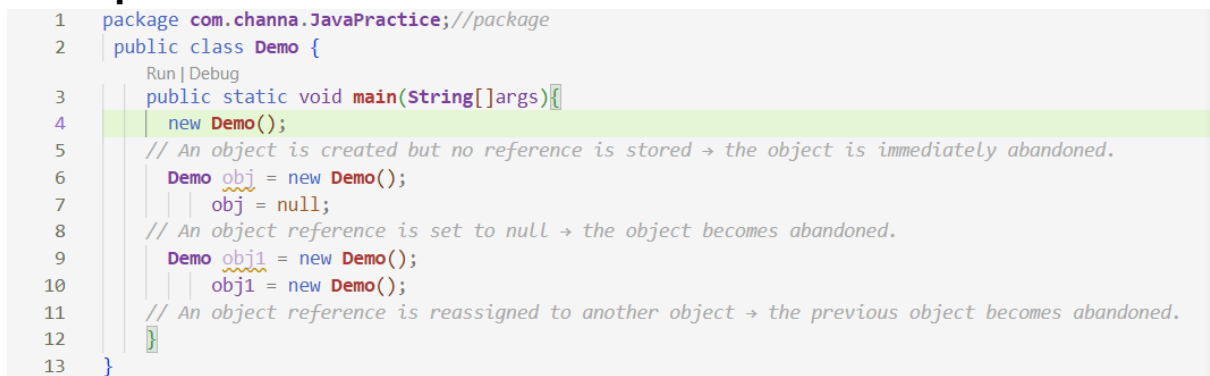
```
1 package com.channa.JavaPractice; //package
2 public class Demo {
3     public static void main(String[] args) {
4         Demo s1 = new Demo();
5         Demo s2 = s1;
6     }
7 }
```

➤ Abandoned Object:



An **abandoned object** is also called an **unused object**. That object is **no longer referenced or used by the program**.

Example:



```
1 package com.channa.JavaPractice; //package
2 public class Demo {
3     public static void main(String[] args) {
4         new Demo();
5         // An object is created but no reference is stored → the object is immediately abandoned.
6         Demo obj = new Demo();
7         obj = null;
8         // An object reference is set to null → the object becomes abandoned.
9         Demo obj1 = new Demo();
10        obj1 = new Demo();
11        // An object reference is reassigned to another object → the previous object becomes abandoned.
12    }
13 }
```

➤ Memory management:

✿ ————— ✿
Memory management is handled by the **Garbage Collector**, which is a part of the **JVM**.

The **Garbage Collector** automatically removes **abandoned or unused objects** from the **heap**.

It **reclaims the memory** occupied by those **abandoned objects** and makes it **available for new objects**.

➤ Parameterized Constructor:

✿ ————— ✿
A **parameterized constructor** is a constructor that takes **one or more parameters** to **initialize an object with specific values at the time of object creation**.

Syntax: `className(parameters) {
 // constructor body
 }`

Note: Here, **constructorName** must be the **same as the class name**, and **parameters** are the **variables that accept values** at the time of **object creation**.

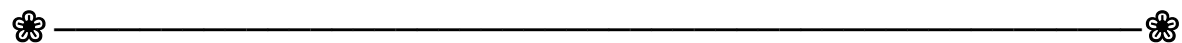
Example:



```
src > com > channa > JavaPractice > J Demo.java > Demo > main(String[])
1 package com.channa.JavaPractice; //package
2 public class Demo {
3     String employeeName; // Instance variable
4     int employeeAge; // Instance variable
5     Demo(String name, int age) {
6         employeeName = name;
7         employeeAge = age;
8     }
9     void display(){
10        System.out.println("Employee name: " + employeeName);
11        System.out.println("Employee age: " + employeeAge);
12    }
13    Run | Debug
14    public static void main(String[] args){
15        //Demo obj = new Demo(); CTE The constructor Demo() is undefined
16        Demo myObj = new Demo(name: "Channakeshavareddy G N", age: 22);
17        myObj.display();
18    }
19 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & "C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe" "-Xx:+showCodeDetailsInExceptionMessages" "-cp" "C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin" "com.channa.JavaPractice.Demo"
Employee name: Channakeshavareddy G N
Employee age: 22
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

➤ No-Argument Constructor:



A **no-argument constructor** is a constructor that **does not take any parameters**.

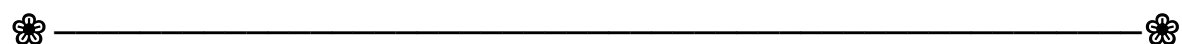
It is used to **initialize objects with default values** and to **create objects without passing any arguments**.

Example:

```
1 package com.channa.javaPractice; //package
2 public class Demo {
3     String employeeName; // Instance variable
4     int employeeAge; // Instance variable
5     //No-argument constructor.
6     Demo() {
7         employeeName = "Channakeshavareddy G N";
8         employeeAge = 22;
9     }
10    void display(){
11        System.out.println("Employee name: " + employeeName);
12        System.out.println("Employee age: " + employeeAge);
13    }
14    public static void main(String[] args){
15        Demo obj = new Demo();
16        obj.display();
17    }
18 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.javaPractice.Demo'
Employee name: Channakeshavareddy G N
Employee age: 22
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

➤ Default Constructor:



A **default constructor** is similar to a **no-argument constructor**, but the difference is that if the programmer **does not define any constructor**, the **Java compiler automatically provides a default constructor** to **initialize the object with default values**.

So, a **default constructor is compiler-provided**, while a **no-argument constructor can be explicitly defined by the programmer**.

Example:

```
1 package com.channa.javaPractice; //package
2 public class Demo {
3     String employeeName; // Instance variable
4     int employeeAge; // Instance variable
5     //No-argument constructor.
6     Demo() {
7         employeeName = "Channakeshavareddy G N";
8         employeeAge = 22;
9     }
10    void display(){
11        System.out.println("Employee name: " + employeeName);
12        System.out.println("Employee age: " + employeeAge);
13    }
14    public static void main(String[] args){
15        Demo obj = new Demo();
16        obj.display();
17    }
18 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.javaPractice.Demo'
Employee name: Channakeshavareddy G N
Employee age: 22
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

Note: The object is created with default values as null & 0 for its instance variables.

Programmer explicitly assigns new values, which update those instance variables.

Therefore, the updated values can be accessed through the object reference.

This Keyword



this is a **reference keyword** in Java that refers to the **current object (current instance)**.

this is commonly used to **differentiate instance variables from local variables or parameters** when they have the same name, **especially in a constructor**.

Example:

```
src > com > channa > JavaPractice > J Demo.java > Demo > Demo(String, int)
1 package com.channa.javaPractice; //package
2 public class Demo {
3     String name; // Instance variable
4     int age; // Instance variable
5
6     Demo(String name, int age) { //parameters:
7         this.name = name;
8         this.age = age;
9     }
10    // The above constructor uses the this keyword to differentiate
11    // instance variables from constructor parameters because both have the same name.
12    void display(){
13        System.out.println("Employee name: " + name);
14        System.out.println("Employee age: " + age);
15    }
16
17    Run | Debug
18    public static void main(String[] args){
19        Demo myObj = new Demo(name: "Channakeshavareddy G N", age: 22);
20        myObj.display();
21    }
22 }
```

Terminal

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.javaPractice.Demo'
Employee name: Channakeshavareddy G N
Employee age: 22
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

Method Chaining



Method chaining is the process of **calling multiple methods** on the same object in a **single statement** by returning **this**, which provides the **current object reference** for the next **method call**

Syntax:

```
ClassName method1() {
    // statements
    return this;
}
```

```
ClassName method2() {
    // statements
    return this;
}
```

Example:

```
1 package com.channa.JavaPractice;
2
3 public class Student {
4
5     String name;
6     int age;
7
8     Student setName(String name) {
9         this.name = name;
10        return this;
11    }
12
13    Student setAge(int age) {
14        this.age = age;
15        return this;
16    }
17
18    void display() {
19        System.out.println("Student name: " + name);
20        System.out.println("Student age: " + age);
21    }
22
23    Run | Debug
24    public static void main(String[] args) {
25        Student obj = new Student().setName(name: "Channakeshavareddy G N").setAge(age: 22);
26        obj.display();
27    }
28 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & "C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe" "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin" "com.channa.JavaPractice.Student"
Student name: Channakeshavareddy G N
Student age: 22
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

Note: In method chaining, every method whose result is used to call another method must return an **object/reference**, commonly **this**.

If a method returns **void**, it cannot be used to continue **method chaining**, so call it separately.

Constructor Overloading



Constructor overloading is the process of **defining multiple constructors within the same class with different parameter lists**.

Constructor overloading is used to **initialize objects in different ways based on the parameters provided**.

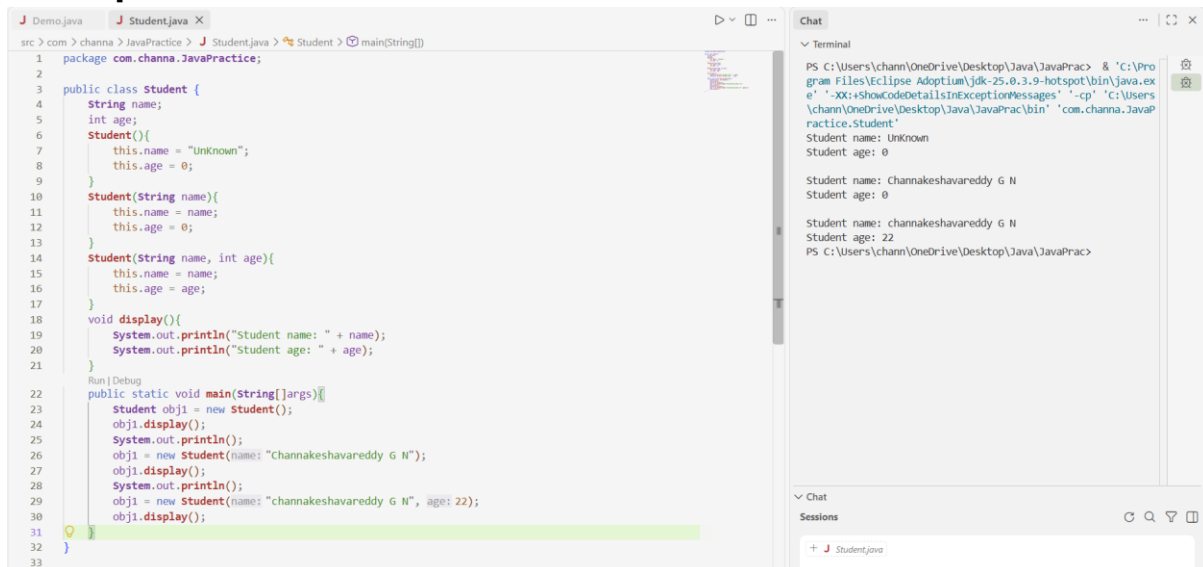
Syntax:

```
class Student {

    Student() {
    }

    Student(int age) {
    }
    Student(int age, String name) {
    }
}
```

Example:



```
1 package com.channa.JavaPractice;
2
3 public class Student {
4     String name;
5     int age;
6     Student(){
7         this.name = "Unknown";
8         this.age = 0;
9     }
10    Student(String name){
11        this.name = name;
12        this.age = 0;
13    }
14    Student(String name, int age){
15        this.name = name;
16        this.age = age;
17    }
18    void display(){
19        System.out.println("Student name: " + name);
20        System.out.println("Student age: " + age);
21    }
22    public static void main(String[] args){
23        Student obj1 = new Student();
24        obj1.display();
25        System.out.println();
26        obj1 = new Student(name: "channakeshavareddy G N");
27        obj1.display();
28        System.out.println();
29        obj1 = new Student(name: "channakeshavareddy G N", age: 22);
30        obj1.display();
31    }
32 }
33 }
```

Terminal Output:

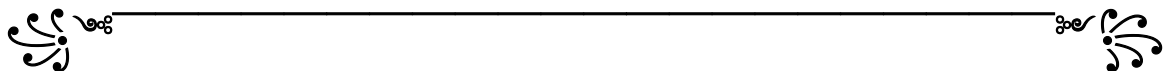
```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin" "com.channa.JavaPractice.Student"
Student name: Unknown
Student age: 0

Student name: channakeshavareddy G N
Student age: 0

Student name: channakeshavareddy G N
Student age: 22
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

Note: obj1 —► Object 1
obj1 —► Object 2
Object 1 → unreachable (Abandoned)
obj1 —► Object 3
Object 2 → unreachable (Abandoned)

Constructor Chaining



Constructor chaining occurs when **one constructor calls another constructor within the same class using the this() keyword.**

Constructor chaining is used to **initialize objects at different levels based on the constructor call.**

Note: this() must be the first statement in constructor.

Syntax:

```
Student() {
    this("Unknown", 0);
}
Student(String name, int age) {
    this.name = name;
    this.age = age;
}
```

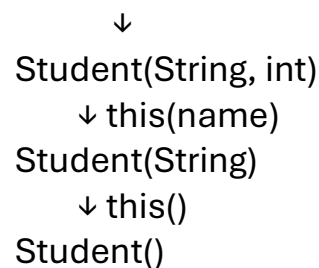
Example:

```
1 package com.channa.JavaPractice;
2
3 public class Student {
4     String name;
5     int age;
6     Student()//signature matched
7     {
8         this.name = "Unknown";
9         this.age = 0;
10    }
11    Student(String name){ //signature matched
12        this(); // constructor called
13        this.name = name;
14        this.age = 0;
15    }
16    Student(String name, int age){ //signature matched
17        this(name);//constructor called
18        this.age = age;
19    }
20    void display(){
21        System.out.println("Student name: " + name);
22        System.out.println("Student age: " + age);
23    }
24    public static void main(String[] args){
25        Student obj1 = new Student(name: "channakeshavareddy G N", age: 22);//constructor called
26        obj1.display();
27    }
28 }
```

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & "C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe" "-XX:+ShowCodeDetailsInExceptionMessages" -cp "C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin" "com.channa.JavaPractice.Student"

Student name: channakeshavareddy G N
Student age: 22
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>

Execution chain: new Student(String, int)



Blocks in Java

 **A block is a group of one or more statements enclosed within curly braces {}.** 

Blocks define their **scope and execution flow**.

Example:

```
1 package com.channa.JavaPractice;
2
3 public class Student {
4     Run | Debug
5     public static void main(String[] args){
6         int age = 10; // statement enclosed with curly braces.
7     }
8 }
```

The main types of blocks in Java are:

1. Static block
2. Instance block
3. Local block

➤ Static Block:



A **static block**, also known as a **static initializer block**, is used to **initialize static variables and perform class-level initialization**.

A **static block** executes when the class is initialized, before the **main()** method executes.

That is why **instance variables cannot be directly accessed inside a static block**, because **no instance is created when the static block executes**.

Syntax:

```
static{  
    //statements  
}
```

Example:

```
1 package com.channa.JavaPractice;  
2  
3 public class Student {  
4     static int a;  
5     static {  
6         a = 10;  
7         System.out.println("Static block 1 executed: " + a);  
8     }  
9     static{  
10        System.out.println("Static block 2 executed.");  
11    }  
12    static{  
13        System.out.println("Static block 3 executed.");  
14    }  
15  
16    public static void main(String[] args){  
17        System.out.println("Main method starts.");  
18        System.out.println("Main method executed.");  
19        System.out.println("Main method ends.");  
20    }  
}
```

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' ^-XX:+ShowCodeDetailsInExceptionMessages ^-cp ^"C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin" ^com.channa.JavaPractice.Student
Static block 1 executed: 10
Static block 2 executed.
Static block 3 executed.
Main method starts.
Main method executed.
Main method ends.
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>

➤ Instance Block:



An **instance block**, also known as an **instance initializer block**, is used to **initialize instance variables**.

An **instance block** executes every time an instance (object) of the class is created.

Static variables can be accessed inside an instance block because static variables belong to the **class** and **share data across all instances**.

Syntax:

```
{  
    //statements  
}
```

Example:

```
1 package com.channa.JavaPractice;
2
3 public class Student {
4     static int a;
5     static {
6         a = 10;
7         System.out.println("Static block executed: " + a);
8     }
9     {
10        System.out.println(x: "Instance block 1 executed.");
11    }
12    {
13        System.out.println(x: "Instance block 2 executed.");
14    }
15    public static void main(String[] args) {
16        Student obj = new Student();
17        System.out.println(x: "Main method starts.");
18        System.out.println(x: "Main method ends.");
19    }
20 }
21
```

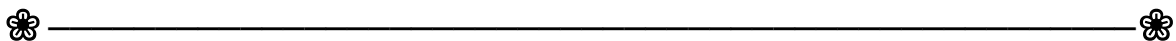
```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & "C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe" "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin" "com.channa.JavaPractice.Student"
Static block executed: 10
Instance block 1 executed.
Instance block 2 executed.
Main method starts.
Main method ends.
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

Note: If a class contains multiple **static blocks** and **instance blocks**, the **static blocks execute from top to bottom first**, followed by the **instance blocks** when an object is created.

If a **static block creates an object**, the **current static block executes**, then the **instance block executes then the constructor**, and the remaining **static blocks** execute before the **main() method**.

Note: Java allows **instance creation inside a static block** because **instance creation does not require an existing instance**.

► In-line initialization:



The variable is initialized exactly at the point of its declaration in the class.

The value must be available at compile time.

Syntax: `int variable = value;`

Example:

```
1 package com.channa.JavaPractice;
2
3 public class Student {
4     Run | Debug
5     public static void main(String[] args){
6         int age = 10; // Inline Initialization.
7     }
8 }
```

Here, age is a variable, you assign **10 at the point of declaration**, So, it is **inline initialization**.

Access Modifiers in Java



Access modifiers determine the **accessibility / visibility** of a **class, method, field, or constructor** in Java.

They control **who can access and use the code**.

Access modifiers are classified into 4 types:

1. **private:** Accessible **only within the same class**.
2. **default (no modifier):** Accessible **within the same package**.
3. **protected:** Accessible **within the same package**, and its **subclasses**, even if the subclass are in **different package**.
4. **public:** Accessible from **any class, any package, anywhere in the program**.

Note: **public** members have the **highest level of accessibility / visibility**.

Core Principals of OOP in Java



- ⇒ **Encapsulation.**
- ⇒ **Inheritance.**
- ⇒ **Polymorphism.**
- ⇒ **Abstraction.**

Encapsulation in Java



Encapsulation is the process of **binding attributes and methods** into a single class and **hiding the data members** using the **private access modifier**.

We can access those **private members** from outside the class through **getters (accessors)** and **setters (mutators)**.

Example:

```
1 package com.channa.JavaPractice; //package
2 class Student {
3     // Attributes
4     private String name;
5     private int age;
6     // Method
7     void display() {
8         System.out.println("Name: " + name);
9         System.out.println("Age: " + age);
10    }
11    // Setter (Mutator)
12    public void setName(String name) {
13        this.name = name;
14    }
15    public void setAge(int age) {
16        this.age = age;
17    }
18    // Getter (Accessor)
19    public String getName() {
20        return name;
21    }
22    public int getAge() {
23        return age;
24    }
25 }
26 public class Demo {
27     public static void main(String[] args) {
28         Student obj = new Student();
29         obj.setName(name: "Channakeshavareddy g N");
30         System.out.println("My Name is: " + obj.getName());
31         System.out.println();
32         obj.setAge(age: 22);
33         System.out.println("My age is : " + obj.getAge());
34     }
35 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
My Name is: Channakeshavareddy g N

My age is : 22
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

Key idea: Binding + Data Hiding + Controlled Access = Encapsulation

➡ Data Encapsulation:



Data Encapsulation is the process of **hiding the attributes of a class** using the **private** access and accessing them indirectly from outside the class through **getters and setters**.

In **Data Encapsulation**, member **methods** exist to **access and modify the hidden data members**.

Types of Methods in Encapsulation

1. Mutator Methods (Setters)
2. Accessor Methods (Getters)

↳ Mutator methods:



Mutator methods (Setters) Their purpose is to **modify the value of a private variable** from **outside the class**.

Mutator methods are **commonly declared as public**, so that the private variable can be **updated through controlled access**.

Example:

```
1 package com.channa.JavaPractice;//package
2 class Student {
3     // Attributes
4     private String name;
5     private int age;
6     // Method
7     void display() {
8         System.out.println("Name: " + name);
9         System.out.println("Age: " + age);
10    }
11    // Setter (Mutator)
12    public void setName(String name) {
13        this.name = name;
14    }
15    public void setAge(int age) {
16        this.age = age;
17    }
18 }
19 public class Demo {
20     Run | Debug
21     public static void main(String[] args){
22         Student obj = new Student();
23         obj.setName(name: "Channakeshavareddy g N");
24         obj.setAge(age: 22);
25     }
26 }
```

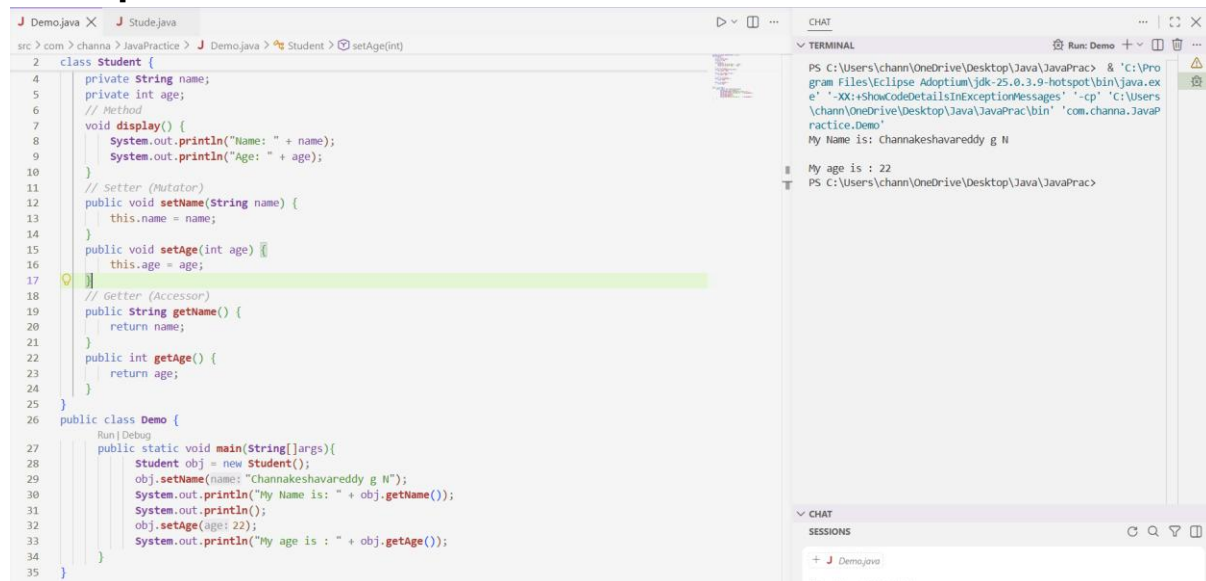
↳ Accessor methods:



Accessor Methods (Getters) Their purpose is to **read or get the value of a private variable** from **outside the class**.

Accessor methods are **commonly declared as public**, so that the private variable can be **accessed through controlled access**.

Example:



```
src > com > channa > JavaPractice > J Demo.java > Student > setAge(int)
2 class Student {
3     private String name;
4     private int age;
5     // Method
6     void display() {
7         System.out.println("Name: " + name);
8         System.out.println("Age: " + age);
9     }
10
11     // Setter (Mutator)
12     public void setName(String name) {
13         this.name = name;
14     }
15     public void setAge(int age) {
16         this.age = age;
17     }
18     // Getter (Accessor)
19     public String getName() {
20         return name;
21     }
22     public int getAge() {
23         return age;
24     }
25 }
26 public class Demo {
27     public static void main(String[] args) {
28         Student obj = new Student();
29         obj.setName(name: "Channakeshavareddy g N");
30         System.out.println("My Name is: " + obj.getName());
31         System.out.println();
32         obj.setAge(age: 22);
33         System.out.println("My age is : " + obj.getAge());
34     }
35 }
```

CHAT

TERMINAL

Run: Demo

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin" "com.channa.JavaPractice.Demo"

My Name is: Channakeshavareddy g N

My age is : 22

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>

CHAT

SESSIONS

+ J Demo.java

Inheritance in Java



Inheritance is the process by which a **subclass** acquires the properties and behaviours of a **superclass**.

In inheritance, classes are categorized into 2 types:

1. Subclass
2. Superclass.

Subclass: A class that **acquires** the properties and behaviours from a **superclass**.

Superclass: A class from which a **subclass** acquires the properties and behaviours.

↳ **extend keyword:**



The **extend keyword** define the **relationship** between **subclass** and **superclass**.

Syntax: subclass **extends** superclass

Inheritance is categorized into 5 types:

1. **Single** Inheritance.
2. **Multiple** Inheritance
3. **Multilevel** Inheritance.
4. **Hierarchical** Inheritance.
5. **Hybrid** Inheritance.

➤ Single Inheritance:



Single Inheritance A subclass inherits the properties and behaviours from only one superclass.

Example:

```
1 package com.channa.JavaPractice; //package
2 class Animal{ //Superclass
3     void showAnimal(){
4         System.out.println("Animal is moving.");
5     }
6 }
7 class Dog extends Animal{ //Subclass
8     void showDog(){
9         System.out.println("Dog is barking.");
10    }
11 }
12 public class Demo {
13     public static void main(String[] args){
14         Dog obj = new Dog(); //object created for subclass
15         obj.showDog(); //subclass own methods
16         obj.showAnimal(); //subclass inheritd method.
17     }
18 }
```

TERMINAL

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin" "com.channa.JavaPractice.Demo"
Dog is barking.
Animal is moving.
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

↳ Multiple Inheritance:



A subclass inherits properties and behaviours from more than one superclass.

Java does not support multiple inheritance through classes because of the **Diamond Problem**, but multiple inheritance can be achieved through interfaces.

Note: We will learn Multiple Inheritance in Interfaces.

↳ Dimond problem in Multiple Inheritance:

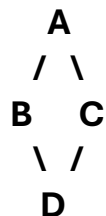


The **Diamond Problem** is an **ambiguity error** that occurs in **multiple inheritance** when a **subclass extends multiple superclasses**.

But

Java prevents this by not supporting multiple inheritance through classes.

The classic **Diamond Problem** is:



Meaning:

- ↳ B extends A
- ↳ C extends A
- ↳ D would extend both B and C

So:

A → B, C → D creates the **Diamond Problem**.

The issue is that **D inherits from B and C, and both ultimately inherit from A**, which can create ambiguity about which inherited implementation **D** should use.

Channakeshavareddy G N

But in Java:

**class D extends B, C { // ❌ Compile-time error
}**

is not allowed.

Example:

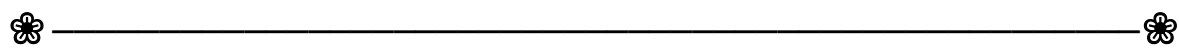
```
1 package com.channa.JavaPractice;
2
3 class Dog{
4     void showDog(){
5         System.out.println("Dog Barking.");
6     }
7 }
8 class Cat{
9     void showCat(){
10        System.out.println("Cat Meows.");
11    }
12 }
13 class Animal extends Cat, Dog{ //Compile-time error
14     void showAnimal(){
15         System.out.println("Animal making sound");
16     }
17 }
18 public class Demo1 {
19     public static void main(String[] args) {
20         Animal obj = new Animal();
21         obj.showDog();
22         obj.showCat(); //animal cannot extend cat here so error
23         obj.showAnimal();
24     }
25 }
```

PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'

Exception in thread "main" java.lang.Error: Unresolved compilation problems:
The method showDog() is undefined for the type Animal
The method showCat() is undefined for the type Animal

at com.channa.JavaPractice.Demo1.main(Demo1.java:21)
PS C:\Users\chann\OneDrive\Desktop\Java>

➡ Multilevel Inheritance:



The process in which a **subclass inherits from a superclass**, and that **superclass itself inherits from another superclass**.

Class chain: Grandparent → Parent → Child

Example:

```
1 package com.channa.JavaPractice; //package
2 class Animal{ //Superclass
3     void showAnimal(){
4         System.out.println("Animal is moving.");
5     }
6 }
7 class Dog extends Animal{ //Subclass
8     void showDog(){
9         System.out.println("Dog is barking.");
10    }
11 }
12 class PuppyDog extends Dog{ //Subclass
13     void showPuppy(){
14         System.out.println("Puppy Dog is playing.");
15     }
16 }
17 public class Demo {
18     public static void main(String[] args){
19         PuppyDog obj = new PuppyDog(); //object created for subclass
20         obj.showAnimal(); //subclass inherited method.
21         obj.showDog(); //subclass inherited methods
22         obj.showPuppy(); //subclass its own method
23     }
24 }
```

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'

Animal is moving.
Dog is barking.
Puppy Dog is playing.

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>

► Hierarchical Inheritance:



Hierarchical Inheritance is the process by which **multiple subclasses** inherit from a **single superclass**.

In **Hierarchical Inheritance**, we typically create an **object for each subclass** when we want to access the **specific behaviours of each subclass**, because multiple subclasses inherit from the **same superclass**.

Example:

```
src > com > channa > JavaPractice > J Demo.java > Dog > showDog()
1 package com.channa.JavaPractice; //package
2 class Animal{
3     void showAnimal(){ //Superclass
4         System.out.println("Animal is moving.");
5     }
6 }
7 class Dog extends Animal{ //Subclass
8     void showDog(){
9         System.out.println("Dog is barking.");
10    }
11 }
12 class PuppyDog extends Animal{ //Subclass
13     void showPuppy(){
14         System.out.println("Puppy Dog is playing.");
15    }
16 }
17 public class Demo {
18     Run | Debug
19     public static void main(String[] args){
20         PuppyDog obj1 = new PuppyDog(); //object
21         obj1.showAnimal();
22         obj1.showPuppy();
23         Dog obj2 = new Dog();
24         obj2.showAnimal();
25         obj2.showDog();
26     }
27 }
```

CHAT

TERMINAL

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
Animal is moving.
Puppy Dog is playing.
Animal is moving.
Dog is barking.
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

► Hybrid Inheritance:



Hybrid Inheritance is a combination of two or more types of inheritance.

Note: Java does not support **hybrid inheritance** through **classes** when the structure requires **multiple inheritance of classes**.

Example: Multilevel + Hierarchical = Hybrid Inheritance.

```
src > com > channa > JavaPractice > J Demo.java > Demo > main(String[])
1 package com.channa.JavaPractice; //package
2 class Animal{
3     void showAnimal(){ // Superclass
4         System.out.println("Animal is moving.");
5     }
6 }
7 class Dog extends Animal{ // Subclass
8     void showDog(){
9         System.out.println("Dog is barking.");
10    }
11 }
12 class Cat extends Dog{ // subclass
13     void showCat(){
14         System.out.println("Cat is meowing.");
15    }
16 }
17 class PuppyDog extends Dog{ // Subclass
18     void showPuppy(){
19         System.out.println("Puppy Dog is playing.");
20    }
21 }
22 public class Demo {
23     Run | Debug
24     public static void main(String[] args){
25         PuppyDog obj1 = new PuppyDog(); // single
26         obj1.showDog();
27         obj1.showPuppy();
28         obj1.showAnimal();
29         Cat obj2 = new Cat(); // multilevel
30         obj2.showAnimal();
31         obj2.showDog();
32         obj2.showCat();
33     }
34 }
```

CHAT

TERMINAL

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
Dog is barking.
Puppy Dog is playing.
Animal is moving.
Animal is moving.
Dog is barking.
Cat is meowing.
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

CHAT

SESSIONS

+ J Demo.java

Describe what to build

Super Keyword



super is a reference keyword used to **access the instance variables and instance methods and constructor of the superclass.**

Both **this()** and **super()** must be the **first statement inside a constructor.**

Example:

```
1 package com.channa.JavaPractice; //package
2 class Student{
3     private String name;
4     private String ID;
5     public Student(String name, String ID){
6         this.name = name;
7         this.ID = ID;
8     }
9     public String getName(){
10        return name;
11    }
12    public String getID(){
13        return ID;
14    }
15 }
16 class Age extends Student{
17     private int age; // Subclass
18     public Age(String name, String ID, int age){
19         super(name, ID);
20         this.age = age;
21     }
22     public int getAge(){
23         return age;
24     }
25 }
26 public class Demo {
27     Run | Debug
28     public static void main(String[] args){
29         Age obj = new Age(name: "Channakeshavareddy G N", ID: "25MCAR117", age: 22);
30         System.out.println(obj.getName());
31         System.out.println(obj.getID());
32         System.out.println(obj.getAge());
33     }
34 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
Channakeshavareddy G N
25MCAR117
22
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

Difference Between this and super?

this	Super
this refers to the current class object.	super refers to the immediate parent class.
this is used to access current class variables, methods, and constructors.	super is used to access parent class variables, methods, and constructors.

Note: The method is called by which instance is called current instance.

Difference Between Constructor and Method?

Constructor	Method
Constructor is used to initialize an object.	Method is a reusable block of code used to perform an operation.
Constructor has the same name as the class and has no return type.	Method can have any valid name and must have a return type or void.

Polymorphism in Java



Polymorphism is the **ability of an object to take many forms** and **perform different behaviours.**



Polymorphism is categorized into 2 types:

1. Compile-time polymorphism
2. Runtime polymorphism

➤ Compile-time Polymorphism:



Compile-time polymorphism is achieved through **method overloading.**



Method overloading occurs when a class has **multiple methods with the same name but different parameter lists.**

Method overloading is used to perform **the same functionality with different types of parameters.**

Example:

```
1 package com.channa.JavaPractice; //package
2 public class Demo {
3     int add(int num1, int num2) {
4         return num1 + num2;
5     }
6     int add(int num1, int num2, double num3){
7         return num1 + num2 + (int)num3;
8     }
9     Run | Debug
10    public static void main(String[] args){
11        Demo obj = new Demo();
12        System.out.println(obj.add(10, 20));
13        System.out.println(obj.add(10,20,30.5));
14    }
}
```

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
30
60
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```


➤ Run-time Polymorphism:



Runtime polymorphism is achieved through **method overriding**.

Method overriding occurs when a method is defined in different classes with the **same method signature**.

it is used to perform **the same functionality with different implementations**.

Example:

```
1 package com.channa.JavaPractice; //package
2
3 class Animal{
4     void sound(){
5         System.out.println("Animal makes sound.");
6     }
7 }
8 class Dog extends Animal{
9     @Override
10    void sound(){
11        System.out.println("Dog is barking.");
12    }
13 }
14 class cat extends Dog{
15     @Override
16    void sound(){
17        System.out.println("Cat meows.");
18    }
19 }
20 public class Demo {
21     Run|Debug
22     public static void main(String[] args){
23         Dog obj = new Dog();
24         obj.sound();
25     }
26 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
Dog is barking.
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>
```

Binding in Java



Binding is the process of connecting a **method call** with the **method implementation**.

Binding is classified into **two types**:

- **Early Binding**
- **Late Binding**

↳ Early Binding:



Early Binding is the process of connecting a **method call** with its **actual method implementation at compile time**.

In Early Binding, the **compiler determines exactly which method or variable to access at compile time** based on the **reference type**.

Early Binding is also known as **Static Binding** because the **compiler determines which method to call at compile time based on the reference type and method signature**, so the method implementation is fixed at compile time.

In Java, **data members (fields)** are bound at **compile time** based on the **reference type**.

Early Binding is achieved through **method overloading**, **static methods**, **final methods**, and **private methods**.

“Because”

- ↳ **Method overloading**: compiler determines the method based on the **method signature**.
- ↳ **Static methods**: bound to the **class**, not the runtime object.
- ↳ **Final methods**: cannot be overridden, so the implementation is fixed.
- ↳ **Private methods**: are not inherited/overridden, so the implementation is fixed.

Example:

```
1 package com.channa.javaPractice; //package
2
3 public class Demo {
4     private int add(int num){
5         return num + 5;
6     }
7     static int add(int num1, int num2){
8         return num1 + num2;
9     }
10    final int add(int num1, int num2, int num3){
11        return num1 + num2 + num3;
12    }
13    Run | Debug
14    public static void main(String[] args){
15        Demo obj = new Demo();
16        System.out.println(obj.add(num: 5));
17        System.out.println(add(5, 20));
18        System.out.println(obj.add(5, 6, 9));
19    }
20
21 PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-X
22 PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:
23 +ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334
24 087bb01b694a44c\redhat-java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.javaPractice.Demo'
25
26
27 PS C:\Users\chann\OneDrive\Desktop\Java>
```

➤ Late Binding:

✿ Late Binding is the process of connecting a **method call** with its **actual method implementation at runtime**.

In Late Binding, the **JVM determines at runtime which overridden method to call based on the actual object**.

Late Binding is achieved through **method overriding** of instance methods.

Late Binding is also known as **Dynamic Binding** because the **overridden method is determined at runtime based on the actual object**, so the method implementation can vary depending on the object.

Example:

```
1 package com.channa.JavaPractice;
2 class Animal{
3     void show(){
4         System.out.println("Animal making Sound");
5     }
6 }
7 class Dog extends Animal{
8     @Override//Annotation
9     void show(){
10        System.out.println("Dog is Barking");
11    }
12 }
13 public class Demo1 {
14     Run|Debug
15     public static void main(String[] args){
16         Animal myAnimal = new Dog();//actual object is Dog.
17         myAnimal.show();//The overridden method is executed based on the actual object at runtime.
18     }
19 }
```

PS C:\Users\chann\OneDrive\Desktop\Java> & "C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe" -XX:+ShowCodeDetailsInExceptionMessages -cp "C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin" "com.channa.JavaPractice.Demo1"

Dog is Barking
PS C:\Users\chann\OneDrive\Desktop\Java>

Method hiding in Java

In Java, method hiding occurs when a subclass defines a **static method with the same signature** as its superclass.

In method hiding, **static methods are accessed using the class name** to clearly specify which version is being called.

Note: We cannot use `@Override` for method hiding because static methods are hidden, not overridden.

Example:

```
1 package com.channa.JavaPractice;//package
2
3 class Animal{
4     static void display(){
5         System.out.println("Animal makes sound.");
6     }
7 }
8 class Dog extends Animal{
9     // @Override We cannot use @Override for method hiding because static methods are hidden, not overridden
10    static void display(){
11        System.out.println("Dog is barking.");
12    }
13 }
14 public class Demo {
15     Run|Debug
16     public static void main(String[] args){
17         Animal.display();
18         Dog.display();
19     }
20 }
```

PS C:\Users\chann\OneDrive\Desktop\Java> & "C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe" -XX:+ShowCodeDetailsInExceptionMessages -cp "C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin" "com.channa.JavaPractice.Demo"

Animal makes sound.
Dog is barking.
PS C:\Users\chann\OneDrive\Desktop\Java>

@Annotations



@Annotations are **metadata in Java** that provide additional information about code to the **compiler, JVM, or frameworks**.
Syntax: @annotation_name.

Example: @Override.

```
1 package com.channa.JavaPractice;
2 class Animal{
3     void show(){
4         System.out.println("Animal making Sound");
5     }
6 }
7 class Dog extends Animal{
8     @Override
9     void show(){
10        System.out.println("Dog is Barking");
11    }
12 }
13 public class Demo1 {
14     Run | Debug
15     public static void main(String[] args){
16         Animal myAnimal = new Dog();//actual object is Dog.
17         myAnimal.show();//here the only one overridden method execute based on actual object.
18     }
19 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse A
doptium\jdk-25.0.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExcep
tionMessages' -cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspace
Storage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7
ee\bin' 'com.channa.JavaPractice.Demo1'
Dog is Barking
PS C:\Users\chann\OneDrive\Desktop\Java>
```

final Keyword



The **final** keyword is used to **restrict modification**.

We can use the final keyword with:

1. Variables.
2. Methods.
3. Classes.

final variable:



A variable which is **prefixed with the final keyword** is called a **final variable**.

Note: final variables do not have default values; explicit initialization is required before use.

Example:

```
1 package com.channa.JavaPractice;//package
2 public class Demo {
3     Run | Debug
4     public static void main(String[] args){
5         final int a = 10;
6         a = 20;// CTE because final keyword restrict modification.
7     }
8 }
```

↳ Constants:



A variable that is **declared** with the **static** and **final keywords** inside a **class** and **outside any method** is called a **constant**.

Example:

```
1 package com.channa.JavaPractice;//package
2 public class Demo {
3     static final int a = 10;//constant
4     public static void main(String[] args){
5     }
6 }
```

↳ Blank final field:



A **blank final field** is a **final instance variable** that is declared but **not initialized at the time of declaration**.

Example:

```
1 package com.channa.JavaPractice;
2
3 public class Student {
4     final String name;
5     Student(String name){
6         this.name = name;
7     }
8     void display(){
9         System.out.println(name);
10    }
11    public static void main(String[] args){
12        Student std = new Student("Channakeshavareddy G N");
13        std.display();
14    }
15 }
```

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Student'
Channakeshavareddy G N
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>

↳ final method:



A method which is **declared with the final keyword** is called a **final method**.

A **final method cannot be overridden**, but an **overridden method** can be declared as **final** in the **child class**.

```
1 package com.channa.JavaPractice;//package
2 class Animal{
3     void display(){
4         System.out.println("Animal is moving.");
5     }
6 }
7 class Dog extends Animal{
8     @Override
9     final void display(){
10        System.out.println("Dog is barking.");
11    }
12 }
13 public class Demo {
14     public static void main(String[] args){
15         Dog obj = new Dog();
16         obj.display();
17     }
18 }
```

PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\OneDrive\Desktop\Java\JavaPrac\bin' 'com.channa.JavaPractice.Demo'
Dog is barking.
PS C:\Users\chann\OneDrive\Desktop\Java\JavaPrac>

↳ final class:



A **class** which is prefixed with the **final keyword** is called a **final class**.

A **final class cannot be a super class** because the **final keyword** prevents **inheritance**, so no **subclass** can be created.

Example:

```
1 package com.channa.JavaPractice;//package
2
3 final class Animal{ // final superclass
4     void sound(){
5         System.out.println(x: "Animal makes sound.");
6     }
7 }
8 class Dog extends Animal{ //subclass cannot extend
9     @Override
10    void sound(){
11        System.out.println(x: "Dog is barking.");
12    }
13 }
14 public class Demo {
15     Run | Debug
16     public static void main(String[] args){
17         Dog obj = new Dog();
18         obj.sound();
19     }
}
```

Has – A – Relationship in Java



HAS-A relationship means where **one class contains or owns an object of another class**.

HAS-A relationship is also known as a **container-contained relationship** or **whole-part relationship**.

In Java **HAS-A relationship** is categorized into **aggregation** and **composition** based on the **levels of dependency** between the **whole and part objects**.

↳ Composition in Has-A-Relationship:



Composition is a form of **HAS-A relationship**. It represents a **strong association** between **whole** and **part objects**.

The **whole object** owns and manages the **part object**. So, the **part object** cannot exist independently without the **whole object**.

Composition can be achieved through declaring the **part object** as a member variable of the **whole object** and **instantiating the part object within the whole object**.

Example:

```
1 package com.channa.JavaPractice; //package
2 class Engine{
3     private int cc;
4     Engine(int cc){
5         this.cc = cc;
6     }
7     void display(){
8         System.out.println("Engine CC: " + cc);
9     }
10 }
11 class Car{
12     private String carName;
13     private Engine engine;
14     Car(int cc, String carName){
15         this.carName = carName;
16         engine = new Engine(cc);
17     }
18     void startEngine(){
19         engine.display();
20         System.out.println(carName);
21     }
22 }
23 public class Demo {
24     Run | Debug
25     public static void main(String[] args){
26         Car obj = new Car(cc: 4395, carName: "B . M . W");
27         obj.startEngine();
28     }
29 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp "C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\56c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin" "com.channa.JavaPractice.Demo"
Engine CC: 4395
B . M . W
PS C:\Users\chann\OneDrive\Desktop\Java>
```

➤ Aggregation in Has-A-Relationship:



Aggregation is a form of **HAS-A relationship**. It represents a **weak association** between the **whole** and **part objects**.

In aggregation the **part objects** have their own **lifecycle**; they can **create**, **update**, and **delete** independently without relying on the **whole object**.

Example:

```
1 package com.channa.JavaPractice; //package
2 class Engine{
3     private int cc;
4     Engine(int cc){
5         this.cc = cc;
6     }
7     void display(){
8         System.out.println("Engine CC: " + cc);
9     }
10 }
11 class Car{
12     private String carName;
13     private Engine engine;
14     Car(Engine engine, String carName){
15         this.carName = carName;
16         this.engine = engine;
17     }
18     void startEngine(){
19         engine.display();
20         System.out.println(carName);
21     }
22 }
23 public class Demo {
24     public static void main(String[] args){
25         Engine mycc = new Engine(cc: 4395);
26         Car mycar = new Car(mycc, carName: "B . M . W");
27         mycar.startEngine();
28     }
29 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse A
doptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExcep
tionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspace
Storage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdk_ws\Java_e2daa7
ee\bin' 'com.channa.JavaPractice.Demo'
Engine CC: 4395
B . M . W
PS C:\Users\chann\OneDrive\Desktop\Java>
```

Non – Primitive Type Casting



Non-Primitive Type Casting means **converting** a **reference** of **one class type** into a **reference** of **another related class type**.

In Java **Non-Primitive type casting** is categorized into **2 main types**:

1. Upcasting.
2. Downcasting.

↳ Upcasting:



Upcasting is the process of assigning a **subclass object** to a **superclass reference variable**.

To achieve upcasting, an **IS-A relationship** is mandatory.

Upcasting is an **implicit or auto upcasting** because when we assign a **subclass object** to the **superclass reference variable**, the **upcasting** occurs **implicitly**.

By using upcasted reference variable we cannot access **subclass-specific properties**.

Upcasting does not require an **explicit cast** and does not cause **ClassCastException** because a **subclass object** is always an **instance of its superclass** due to the **IS-A relationship**.

Example:

```
1 package com.channa.javaPractice;
2 class StudentDetails{
3     private String name;
4     private String id;
5     private int age;
6     protected StudentDetails(String name, String id, int age){
7         this.name = name;
8         this.id = id;
9         this.age = age;
10    }
11    void showStudentDetails(){
12        System.out.println("Student Name : " + name);
13        System.out.println("Student ID : " + id);
14        System.out.println("Student Age : " + age);
15    }
16 }
17 class University extends StudentDetails{
18     private String universityName;
19     private String department;
20
21     protected University(String name, String id, int age, String universityName, String department){
22         super(name, id, age);
23         this.universityName = universityName;
24         this.department = department;
25     }
26     void showUniversityDetails(){
27         System.out.println("University name : " + universityName);
28         System.out.println("Department : " + department);
29     }
30 }
31 public class Student {
32     Run | Debug
33     public static void main(String[] args){
34         StudentDetails myStud = new University(name: "channakeshavareddy G N", id: "25MKAR117", age: 22,
35         universityName: "Garden City University", department: "School of Computational Science");
36         myStud.showStudentDetails();
37         //myStud.showUniversityDetails(); // CTE
38         //By using up casted reference variable we cannot access sub class specific properties directly.
39     }
40 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-23.9-hotspot\bin\java.exe' "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\n\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334887bb01b694ad4c\redhat\jdk\src\java_23\src\bin" "com.channa.javaPractice.Student"
Student Name : channakeshavareddy G N
Student ID : 25MKAR117
Student Age : 22
PS C:\Users\chann\OneDrive\Desktop\Java>
```

instanceof in Java



instanceof is a **keyword** used as a **type-comparison operator** in Java.

It checks whether an object is an **instance of a particular class or type**.

instanceof is commonly used to perform **safe downcasting** and avoid **ClassCastException** by checking the **object's actual type before casting**.

➤ **The instanceof Operator returns false:**

- when the **reference is null**. /
- when the **actual object is a superclass type but is checked against a subclass**. /
- when the class types are **not related through an IS-A relationship / interface relationship**.

Syntax: if(type instanceof object){}else{}

Example:

```
1 package com.channa.JavaPractice;
2 class StudentDetails{
3     private String name;
4     private String id;
5     private int age;
6     protected StudentDetails(String name, String id, int age){
7         this.name = name;
8         this.id = id;
9         this.age = age;
10    }
11    void showStudDetails(){
12        System.out.println("Student Name : " + name);
13        System.out.println("Student ID : " + id);
14        System.out.println("Student Age : " + age);
15    }
16 }
17 class University extends StudentDetails{
18     private String universityName;
19     private String department;
20
21     protected University(String name, String id, int age, String universityName, String department){
22         super(name, id, age);
23         this.universityName = universityName;
24         this.department = department;
25     }
26     void showUniDetails(){
27         System.out.println("University name : " + universityName);
28         System.out.println("Department : " + department);
29     }
30 }
31 public class Student {
32     public static void main(String[] args){
33         StudentDetails myStud = new University(name: "channakeshavareddy G N", id: "ZSMCAR117", age: 22,
34         universityName: "Garden City University", department: "School of Computational Science");
35         if(myStud instanceof University){
36             University myUni = (University) myStud;
37             myUni.showStudDetails();
38             myUni.showUniDetails();
39         } else {
40             //By using down casted reference variable we can access both sub class & super class specific properties directly.
41             System.out.println("%s cannot downcast it.", myStud);
42         }
43     }
44 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adopti
um\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessag
es' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e3
76685f5334087b601b694a44c\redhat.java\jdt_vs\Java_e2daa7ee\bin' 'com.channa
.JavaPractice.Student'
Student Name : channakeshavareddy G N
Student ID : ZSMCAR117
Student Age : 22
University name : Garden City University
Department : School of Computational Science
PS C:\Users\chann\OneDrive\Desktop\Java>
```

➤ **ClassCastException:**



ClassCastException occurs at runtime when we try to cast a parent class reference to a subclass type but the actual object is not of that subclass type.

We can avoid **ClassCastException** by using **instanceof** keyword

Down Casting:



Downcasting is the process of converting a **superclass reference** into a **subclass reference**.

In downcasting, the actual object must be of the **target subclass type (or its subclass)**; otherwise, a **ClassCastException** occurs.

Downcasting is performed on a **superclass reference variable** that refers to a **subclass object**.

By using an upcasted reference variable, we cannot access **subclass-specific members**. To access **subclass-specific members**, downcasting is required.

Example:

```
1 package com.channa.JavaPractice;
2 class StudentDetails{
3     private String name;
4     private String id;
5     private int age;
6     protected StudentDetails(String name, String id, int age){
7         this.name = name;
8         this.id = id;
9         this.age = age;
10    }
11    void showStudDetails(){
12        System.out.println("Student Name : " + name);
13        System.out.println("Student ID : " + id);
14        System.out.println("Student Age : " + age);
15    }
16 }
17 class University extends StudentDetails{
18     private String universityName;
19     private String department;
20
21     protected University(String name, String id, int age, String universityName, String department){
22         super(name, id, age);
23         this.universityName = universityName;
24         this.department = department;
25     }
26     void showUniDetails(){
27         System.out.println("University name : " + universityName);
28         System.out.println("Department : " + department);
29     }
30 }
31 public class Student {
32     public static void main(String[] args){
33         StudentDetails myStud = new University("channakeshavareddy G N", "25MCAR117", 22,
34         universityName: "Garden City University", department: "School of Computational Science");
35         if(myStud instanceof University){
36             University myUni = (University) myStud;
37             myUni.showStudDetails();
38             myUni.showUniDetails();
39         } else {
40             System.out.println("cannot downcast it.");
41         }
42     }
43 }
```

PS C:\Users\chann\OneDrive\Desktop\Java> & "C:\Program Files\Eclipse Adopti
um\jdk-25.0.3-9-hotspot\bin\java.exe" -XX:+ShowCodeDetailsInExceptionMessag
es -cp "C:\Users\chann\AppData\Local\Roaming\Code\User\workspaceStorage\5e2c3e3
76685f5334887bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin" "com.channa.
JavaPractice.Student"
Student Name : channakeshavareddy G N
Student ID : 25MCAR117
Student Age : 22
University name : Garden City University
Department : School of Computational Science
PS C:\Users\chann\OneDrive\Desktop\Java>

Note: instanceof checks the actual object type before downcasting to ensure the downcast is safe.

Abstraction in Java



Abstraction is the process of **hiding implementation details** and **exposing only the essential functionality** to the user.

In Java, **abstraction** can be achieved through **abstract classes** and **interfaces**.

➤ Abstract class:



An **abstract class** is a class declared using the **abstract keyword** that can contain **abstract and concrete methods**.

Characteristics of an Abstract Class:

- An abstract class declared with the **abstract keyword**.
- It **cannot be instantiated directly**.
- An abstract class can contain **constructors, instance variables, static variables, abstract methods, concrete methods, static methods, final methods, and instance methods**.
- An abstract class can be **extended by subclasses** using the **extends** keyword.
- A **concrete subclass** must implement all inherited **abstract methods**.
- If a subclass does not implement all abstract methods, that subclass must also be declared **abstract**.
- An **abstract class** can have **zero or more abstract methods** because it does not have to contain an **abstract method**.

Example:

```
1 package com.channa.JavaPractice;
2 abstract class Animal{//Abstract class.
3     abstract void showAnimal();//abstract method.
4     //Note: Abstract methods cannot be declared as static final and private methods.
5 }
6 class Dog extends Animal{//concrete class extends abstract class.
7     @Override//Annotation
8     public void showAnimal(){//overridden method.
9         System.out.println("Animal making Sound.");
10    }
11 }
12 public class Demo1 {
13     public static void main(String[] args){
14         Animal myAnimal = new Dog();
15         myAnimal.showAnimal();
16     }
17 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & "C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe" -XX:+ShowCodeDetailsInExceptionMessages -cp "C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5\e2c2e376685f5334887bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin" "com.channa.JavaPractice.Demo1"
Animal making Sound.
PS C:\Users\chann\OneDrive\Desktop\Java>
```

Note: Java achieves **partial abstraction** through **abstract classes** because an **abstract class** can contain both **abstract methods** (without implementation) and **concrete methods** (with implementation).

↳ Abstract method:



An **abstract method** is a method that is declared using the **abstract** keyword and does not have a **method body**; it simply **ends with a semicolon (;)**.

Abstract methods are used to define **what a subclass must do**, while the subclass decides **how to do it**.

Characteristics of an Abstract method:

- ↳ An **abstract method** is declared using the **abstract** keyword.
- ↳ It contains **only a method declaration**, without an implementation/body.
- ↳ It can exist only inside an **abstract class or interface**.
- ↳ A **concrete subclass** must **override and implement** the abstract method.
- ↳ If the subclass **does not implement abstract methods**, the subclass must also be declared as **abstract**.
- ↳ An abstract method **cannot be private, static, or final**, because it must be overridden by a subclass.

Example:

```
1 package com.channa.javaPractice;
2 abstract class Animal{//Abstract class.
3     abstract void showAnimal();//abstract method.
4     //Note: Abstract methods cannot be declared as static final and private methods.
5 }
6 class Dog extends Animal{//concrete class extends abstract class.
7     @Override//Annotation
8     public void showAnimal(){//overridden method.
9         System.out.println("Animal making Sound.");
10    }
11 }
12 public class Demo1 {
13     Run | Debug
14     public static void main(String[] args){
15         Animal myAnimal = new Dog();
16         myAnimal.showAnimal();
17     }
18 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & "C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe" "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f534087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin" "com.channa.javaPractice.Demo1"
Animal making Sound.
PS C:\Users\chann\OneDrive\Desktop\Java>
```

↳ Concrete class:



A **concrete class** is a class that provides **complete implementation** and can be **instantiated to create objects**.

↳ Concrete method:



A **concrete method** is a method that has a **complete implementation** with a **method body**.

Interfaces in Java



An **interface** is a **reference type** in Java that defines a **contract or blueprint for implementing concrete classes**. A **concrete class must implement the interface** and provide implementations for its abstract methods.

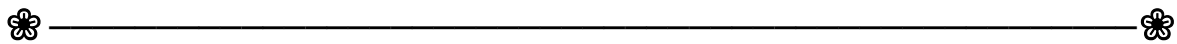
In Java, **Interfaces** are used to achieve **abstraction and multiple inheritance**.

Note: **100% abstraction** can traditionally be achieved using an **interface by declaring only abstract methods** and providing their **implementations in the concrete class**.

Characteristics of an Interface in Java:

- An **Interface** is declared using the **interface keyword**.
- It defines a **contract or blueprint** for concrete classes.
- An interface can contain **abstract, default, static, and private methods**.
- Interface methods are **public and abstract by default** unless explicitly declared as **default, static, or private**.
- Interface variables are **constants** because they are **public, static, and final by default**.
- A **concrete class must implement** the interface using the **implements keyword** and it must provide implementations for **all abstract methods**, otherwise the class must be declared **abstract**.
- An **interface cannot be instantiated directly** because it is a **reference type**, so we can create an **interface reference variable** that can point to an **object of an implementing concrete class**.
- An interface can **extend one or more interfaces** using the **extends keyword**.
- A class can **implement multiple interfaces**, which provides a way to achieve **multiple inheritance** in Java.

☞ Variables in Interfaces:



Interface variables are **constants** because they are **public, static, and final by default**.

- ☞ **Public:** Interface variables are **public**, so they can be **accessed from anywhere** the interface is accessible.
- ☞ **Static:** Interface variables are **static**, so they **belong to the interface itself** and can be accessed using the **interface name**.
- ☞ **Final:** Interface variables are **final**, so their **value cannot be changed** after initialization; they are **constants**.

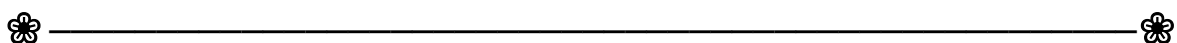
Note: Therefore, interface variables are **public static final by default**.

Example:

```
1 package com.channa.JavaPractice;
2 interface Vehicle {
3
4     int MAX_SPEED = 120;
5
6     void start();
7 }
8
9 class Car implements Vehicle {
10
11     @Override
12     public void start() {
13         System.out.println("Car started");
14     }
15 }
16
17 public class Demo1 {
18     public static void main(String[] args) {
19         System.out.println(Vehicle.MAX_SPEED);
20     }
21 }
22 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0
.3.9-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\ch
ann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat
.java\jdk_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
120
PS C:\Users\chann\OneDrive\Desktop\Java>
```

☞ Default methods in Interfaces:



Default methods are **concrete methods in an interface** that are declared using the **default keyword** and have a **method body**.

They were introduced in **Java 8** to add **new behaviour to interfaces** without breaking existing implementations.

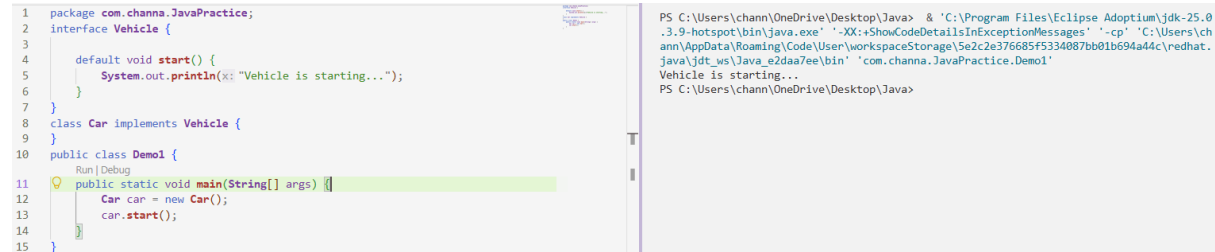
Default methods with the **default keyword** are allowed **only in interfaces** to provide a **default implementation**.

Default methods are **instance methods**, so they can be **invoked using an interface reference variable**.

Note: **default methods** in an **interface** always have **public access implicitly**.

Syntax: **default returnType methodName() {**
 // statements
 }

Example:



```
1 package com.channa.JavaPractice;
2 interface Vehicle {
3
4     default void start() {
5         System.out.println("Vehicle is starting...");
6     }
7 }
8 class Car implements Vehicle {
9 }
10 public class Demo1 {
11     public static void main(String[] args) {
12         Car car = new Car();
13         car.start();
14     }
15 }
```

PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
Vehicle is starting...
PS C:\Users\chann\OneDrive\Desktop\Java>

➤ Static methods in Interfaces:

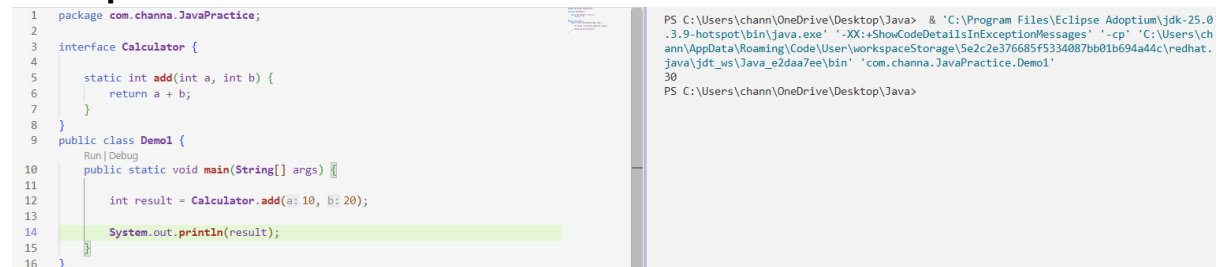
Static methods were introduced in **Java 8** to define **utility or helper methods directly inside an interface**, keeping **related functionality together** and reducing the need for **separate utility classes**.

Static methods belong to the interface itself, so they are invoked using the **interface name**.

Note: static methods in an **interface** always have **public access implicitly**.

Syntax: **static returnType methodName() {**
 // statements
 }

Example:



```
1 package com.channa.JavaPractice;
2
3 interface Calculator {
4
5     static int add(int a, int b) {
6         return a + b;
7     }
8 }
9 public class Demo1 {
10     public static void main(String[] args) {
11         int result = Calculator.add(10, 20);
12         System.out.println(result);
13     }
14 }
15
16 }
```

PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
30
PS C:\Users\chann\OneDrive\Desktop\Java>

➤ Private methods in Interfaces:



Private methods were introduced in **Java 9** to **avoid code duplication** and improve **code reusability** within an interface.

They are mainly used as **helper methods** that can be called by the interface's **default and static methods**.

Private methods are **accessible only within the interface itself**.

Syntax: **private** returnType methodName() {
 // statements
 }

Example:

```
1 package com.channa.JavaPractice;
2
3 interface Vehicle {
4
5     default void start() {
6         checkEngine();
7         System.out.println("Vehicle started");
8     }
9
10    default void stop() {
11        checkEngine();
12        System.out.println("Vehicle stopped");
13    }
14
15    private void checkEngine() {
16        System.out.println("Checking engine...");
17    }
18 }
19
20 class Car implements Vehicle {
21 }
22
23 public class Demo1 {
24     Run | Debug
25     public static void main(String[] args) {
26
27         Car car = new Car();
28
29         car.start();
30         car.stop();
31     }
32 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0
.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ch
anna\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.
java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
Checking engine...
Vehicle started
Checking engine...
Vehicle stopped
PS C:\Users\chann\OneDrive\Desktop\Java>
```

In Java, interfaces are classified into 3 types:

1. Regular Interface
2. Functional Interface
3. Marker Interface

Regular Interface



A **regular interface** is an interface that contains **more than one abstract method**.

It is used when a **contract requires multiple abstract methods**, allowing implementing classes to provide their **own implementations**.

Syntax:

```
interface InterfaceName {  
    returnType methodName1();  
    returnType methodName2();  
    // more abstract methods  
}
```

Example:

```
1 package com.channa.JavaPractice;  
2  
3 interface Parents{ //interface  
4     void showFather();  
5     void showMother();  
6 }  
7 class Son implements Parents{  
8     @Override  
9     public void showFather(){  
10         System.out.println(x: "I am Your Father!");  
11     }  
12     @Override  
13     public void showMother(){  
14         System.out.println(x: "I am Your Mother!");  
15     }  
16 }  
17 public class Demo1 {  
18     Run | Debug  
19     public static void main(String[] args){  
20         Parents myFather = new Son();//Upcasting  
21         myFather.showFather();  
22         myFather.showMother();  
23     }  
24 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0  
.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ch  
ann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.  
java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'  
I am Your Father!  
I am Your Mother!  
PS C:\Users\chann\OneDrive\Desktop\Java>
```

Functional Interface



A **functional interface** is an interface that contains **exactly one abstract method**.

A **functional interface** can be indicated using the **@FunctionalInterface** annotation, which ensures that the interface has **exactly one abstract method**.

Functional interfaces are used to **enable lambda expressions and functional programming in Java**.

Syntax: **@FunctionalInterface**
 interface InterfaceName {
 returnType methodName();
 }

Example:

```
1 package com.channa.JavaPractice;
2 @FunctionalInterface
3 interface Father{ //interface
4     void showFather();
5 }
6 @FunctionalInterface
7 interface Mother{ //interface
8     void showMother();
9 }
10 class Son implements Father{ //Multiple inheritance through interface.
11     @Override
12     public void showFather(){
13         System.out.println("I am Your Father!");
14     }
15 }
16 class Daughter implements Mother{
17     @Override
18     public void showMother(){
19         System.out.println("I am Your Mother");
20     }
21 }
22 public class Demo1 {
23     Run | Debug
24     public static void main(String[] args){
25         Father myFather = new Son();//Upcasting
26         myFather.showFather();
27         Mother myMother = new Daughter();//Upcasting
28         myMother.showMother();
29     }
}
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
```

```
I am Your Father!
I am Your Mother
PS C:\Users\chann\OneDrive\Desktop\Java>
```

Marker Interface



A **marker interface** is an **empty interface** that contains **no methods or variables**. It is also known as a **tag interface**.

A **marker interface** is used to **mark a class**, so that the **JVM or other Java mechanisms can provide special behaviour** to that class.

Syntax: **interface** InterfaceName {
 }

There are several predefined marker interfaces in Java, but the main ones are:

1. **Serializable** : Marks objects for serialization → (we will learn in file handling concepts)
2. **Cloneable** : Marks objects as eligible for cloning.
3. **Remote** : Marks objects as remote objects for Java RMI.

↳ Cloneable:



Cloneable is a **marker interface** used to **mark or tag an object as eligible for cloning**.

Object cloning is the process of **creating a copy of an existing object** using the **clone()** method without **manually copying each field**.

clone() is a **concrete method** in the **Object class**, which is a **class in the java.lang package** and is **automatically imported by the Java compiler**.

The **concrete class's overridden clone()** method calls the **Object class's clone()** method using the **super keyword**, and the **Object class's clone()** method returns the **newly created cloned object** back to the **overridden clone()** method.

Syntax: `return super.clone();`

Syntax: **class** ClassName **implements** Cloneable {
 // class members
 }

Example:

```
1 package com.channa.JavaPractice;
2
3 class Student implements Cloneable { // Marked that student is eligible for cloning.
4
5     int id;
6     String name;
7
8     Student(int id, String name) {
9         this.id = id;
10        this.name = name;
11    }
12
13    // Note: When a method has a class name as its return type,
14    // it can return an object of that class or a subclass of that class.
15    @Override
16    public Object clone() throws CloneNotSupportedException {
17        return super.clone();
18    }
19 }
20
21 public class Demo1 {
22     Run | Debug
23     public static void main(String[] args) throws CloneNotSupportedException {
24
25         Student s1 = new Student(id: 117, name: "Channakeshavareddy G N");
26         // s1 and s2 are two different objects.
27         Student s2 = (Student) s1.clone(); //downcasted to assign s1 object copy to s2 object
28
29         System.out.println(s1.id);
30         System.out.println(s1.name);
31
32         System.out.println(s2.id);
33         System.out.println(s2.name);
34     }
35 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
117
Channakeshavareddy G N
117
Channakeshavareddy G N
PS C:\Users\chann\OneDrive\Desktop\Java>
```

Remote:

Remote is a marker interface used with **Java RMI** to identify **remote objects**.

RMI stands for **Remote Method Invocation**. It is a Java mechanism that allows an object in one JVM to **invoke methods on an object in another JVM**.

Note: RMI is a legacy technology and is not commonly used in modern Java applications, so I focused my efforts on more relevant and widely used technologies.

Multiple inheritance in Java

Multiple inheritance is achieved when a class **implements multiple interfaces**.

Syntax: **class** ClassName **implements** Interface1, Interface2 {
 // implementation of abstract methods

Example:

```
1 package com.channa.JavaPractice;
2
3 interface Father { //interface
4     void showFather();
5 }
6
7 interface Mother { //interface
8     void showMother();
9 }
10
11 class Son implements Father, Mother { //Multiple inheritance through interface.
12     @Override
13     public void showFather() {
14         System.out.println("I am Your Father!");
15     }
16     @Override
17     public void showMother() {
18         System.out.println("I am Your Mother!");
19     }
20 }
21
22 public class Demo1 {
23     Run | Debug
24     public static void main(String[] args) {
25         Father myFather = new Son(); //Upcasting
26         myFather.showFather();
27         Mother myMother = new Son(); //Upcasting
28         myMother.showMother();
29     }
30 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
I am Your Father!
I am Your Mother!
PS C:\Users\chann\OneDrive\Desktop\Java>
```

Wrapper Classes in Java



Wrapper classes are predefined classes provided by the `java.lang` package that wrap primitive data types into objects.

Each primitive data type has a corresponding wrapper class in Java.

↳ <code>int</code>	<code>Integer</code>
↳ <code>char</code>	<code>Character</code>
↳ <code>boolean</code>	<code>Boolean</code>

Wrapper classes are final classes in Java, so we cannot extend or inherit them.

They are immutable, meaning their objects cannot be modified after they are created.

Wrapper classes are used in collections, generics, and whenever objects are required instead of primitive data types.

Wrapper classes convert primitive values into objects and objects back into primitive values through autoboxing and unboxing.

↳ Auto boxing:



Autoboxing is the automatic conversion of a primitive data type into its corresponding wrapper object.

Syntax: `WrapperClass variableName = primitiveValue;`

Example:

```
1 package com.channa.JavaPractice;
2
3 public class Demo1 {
4     public static void main(String[] args) {
5         int num = 10;
6         Integer myWrapper = num;
7         System.out.println(myWrapper.compareTo(anotherInteger: 20));
8         System.out.println("Returns string: " + myWrapper.toString());
9     }
10 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
-1
Returns string: 10
PS C:\Users\chann\OneDrive\Desktop\Java>
```

Note: we will learn wrapper classes internals next collection frameworks.

☛ ➤ Auto Un – boxing:



Unboxing is the **automatic conversion of a wrapper object into its corresponding primitive data type.**

Syntax: `PrimitiveType variableName = wrapperObject;`

Example:

```
1 package com.channa.JavaPractice;
2
3 public class Demo1 {
4     public static void main(String[] args) {
5         int num = 10;
6         Integer myWrapper = num;
7         int unbox = myWrapper;
8         int result = unbox + 2;
9         System.out.println("Result is: " + result);
10    }
11 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' ^-XX:+ShowCodeDetailsInExceptionMessages ^-cp 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'

Result is: 12
PS C:\Users\chann\OneDrive\Desktop\Java>
```

☛ ➤ == vs equals() in wrapper classes:



☛ ➤ == Compares whether **two references refer to the same object.**

Example:

```
1 package com.channa.JavaPractice;
2 import java.util.Scanner;
3 public class Demo1 {
4     public static void main(String[] args) {
5         Scanner scan = new Scanner(System.in);
6
7         String userName = scan.next();
8         String uname = "channa";
9         String password = scan.next();
10        String pass = "Channa@123";
11        if(userName == uname){ // == compares the references belongs to the same object.
12            if(password == pass){
13                System.out.println(x: "Login success");
14            } else {
15                System.out.println(x: "Invalid password");
16            }
17        } else {
18            System.out.println(x: "Provide valid username and password.");
19        }
20        scan.close();
21    }
22 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' ^-XX:+ShowCodeDetailsInExceptionMessages ^-cp 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
channa
channa@123
Provide valid username and password.
PS C:\Users\chann\OneDrive\Desktop\Java>
```

☛ ➤ equals() Compares whether **two objects have the same content/value.**

Example:

```
1 package com.channa.JavaPractice;
2 import java.util.Scanner;
3 public class Demo1 {
4     public static void main(String[] args) {
5         Scanner scan = new Scanner(System.in);
6
7         String userName = scan.next();
8         String uname = "channa";
9         String password = scan.next();
10        String pass = "Channa@123";
11        if(userName.equals(uname)){
12            if(password.equals(pass)){
13                System.out.println(x: "Login success");
14            } else {
15                System.out.println(x: "Invalid password");
16            }
17        } else {
18            System.out.println(x: "Provide valid username and password.");
19        }
20        scan.close();
21    }
22 }
```

```
PS C:\Users\chann\OneDrive\Desktop\Java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3.9-hotspot\bin\java.exe' ^-XX:+ShowCodeDetailsInExceptionMessages ^-cp 'C:\Users\chann\AppData\Roaming\Code\User\workspaceStorage\5e2c2e376685f5334087bb01b694a44c\redhat.java\jdt_ws\Java_e2daa7ee\bin' 'com.channa.JavaPractice.Demo1'
channa
channa@123
Login success
PS C:\Users\chann\OneDrive\Desktop\Java>
```

 **JAVA OOPs COMPLETED** 

Status : ✓ Completed

Foundation : ✓ Strong

Next Module : ⚠ Exception Handling

“One more Java level cleared! 🏆 OOPs— COMPLETED.”
